Python Methods and Libraries Reference for Competitive Programming

Strings, lists, dicts and sets, math, and the standard library modules most used in contests. Companion to the algorithm templates doc. Python 3.10+ assumed; newer-version features are marked.

1. Strings

Strings are immutable, so every method returns a new string.

```
# CASE
s.upper() / s.lower() / s.title() / s.capitalize() / s.swapcase()
s.casefold()                 # aggressive lower, for case-insensitive compare

# TRIM / PAD
s.strip()                    # whitespace (lstrip / rstrip for one side)
s.strip("xy")                # trims ANY of those chars, not the substring
s.zfill(5)                   # '42' -> '00042'
s.center(10, "-")            # ljust / rjust for one side

# SEARCH
s.find("x")                 # index or -1 (-1 is truthy!)
s.rfind("x")                # from the right
s.index("x")                # index or ValueError
s.count("x")                # non-overlapping occurrences
s.startswith("a")           # endswith too; accepts a tuple of options
"x" in s                     # prefer this for membership

# SPLIT / JOIN
s.split(",")                # list
s.split(",", 1)             # maxsplit
s.split()                   # any whitespace, drops empties
s.rsplit(",", 1)
s.splitlines()
s.partition("=")            # (before, sep, after); rpartition from right
"-".join(["a", "b"])        # 'a-b'; called on the separator, items must be str

# REPLACE
s.replace("a", "b")         # all occurrences
s.replace("a", "b", 2)      # first 2 only
s.removeprefix("pre_")      # 3.9+, whole substring
s.removesuffix(".txt")      # 3.9+

# CHECKS (all False on "")
s.isalpha() / s.isdigit() / s.isalnum() / s.isspace()
s.islower() / s.isupper() / s.isascii()

# CONVERT
ord("a")        # 97
chr(97)         # 'a'
ord(c) - ord("a")            # letter -> 0..25
chr(ord("a") + i)            # 0..25 -> letter
```

```
int("101", 2)               # parse binary
list(s)                     # list of chars
"".join(reversed(s))        # or s[::-1]
```

translate

```
t = str.maketrans("abc", "xyz")       # a->x, b->y, c->z (equal lengths)
"aabbcc".translate(t)                 # 'xxyyzz'
str.maketrans("", "", "aeiou")        # 3rd arg = chars to delete
str.maketrans({"&": "&amp;", "b": None})   # dict form: multi-char values, None deletes
```

Formatting

```
f"{name} is {n:.2f}"      # 2 decimals
f"{42:>8}"                # right-align (< left, ^ center)
f"{42:08}"                # zero-pad to width 8
f"{1234567:,}"            # '1,234,567'
f"{0.256:.1%}"            # '25.6%'
f"{255:b}" / f"{255:x}"   # binary / hex, no prefix
f"{name=}"                # debug: "name='Ada'"
```

Indexing and slicing: `s[start:stop:step]`

Start is included, stop is excluded, any part can be omitted. Works the same on lists and tuples.

```
s = "abcdefgh"
s[2:5]     # 'cde'
s[:3]      # 'abc'          from start
s[3:]      # 'defgh'        to end
s[-3:]     # 'fgh'          last 3
s[:-3]     # 'abcde'        all but last 3
s[::2]     # 'aceg'         every 2nd
s[::-1]    # reversed
s[6:1:-1]  # 'gfedc'        negative step: start > stop
s[2:6:-1]  # ''             wrong direction gives empty
```

- Out-of-range slices clamp silently: `"abc"[1:100]` gives `'bc'`.
- `s[:1]` is a safe "first char or empty" (`""[0]` raises).
- `s[-n:]` returns the whole string when `n == 0`. Use `s[len(s)-n:]`.
- Slices are copies; avoid slicing huge strings inside hot loops.
- Strings can't be assigned to by index. Convert to `list(s)`, edit, then `"".join(...)`.

Handy string recipes

```
s == s[::-1]                          # palindrome check
Counter(s)                            # character frequencies
sorted(s) == sorted(t)                # anagram check
"".join(sorted(s))                    # canonical anagram key
[s[i:i+k] for i in range(len(s)-k+1)]  # all substrings of length k
```

2. Lists

Most list methods change the list in place and return `None`.

```
# CREATE
a = []
a = [0] * n                         # n zeros
a = list(range(n))                  # [0..n-1]
a = [x**2 for x in range(5)]        # comprehension
grid = [[0] * m for _ in range(n)] # 2D (NEVER [[0]*m]*n)

# ADD
a.append(x)                         # one item at the end, O(1)
a.extend(iterable)                  # many items at the end
a.insert(i, x)                      # O(n); slow at the front
a += [x, y]                         # same as extend

# REMOVE
a.pop()                             # last item, O(1)
a.pop(i)                            # item at i, O(n)
a.remove(x)                         # first match, ValueError if missing
del a[i]; del a[1:3]
a.clear()

# SEARCH
a.index(x)                          # first index, ValueError if missing
a.count(x)
x in a                              # O(n)

# ORDER
a.sort()                            # in place, returns None
a.sort(reverse=True)
a.sort(key=len)
a.reverse()
sorted(a)                           # NEW list
reversed(a)                         # iterator

# COPY
b = a.copy()                        # shallow (same as a[:] or list(a))
```

Slice assignment

```
a = [1, 2, 3, 4, 5]
a[1:3] = [9]           # [1, 9, 4, 5]; replacement can be a different length
a[1:1] = [7, 8]       # insert without removing
a[:2] = []             # delete first 2
a[::2] = [0, 0, 0]     # extended slice: sizes must match
```

Unpacking and swapping

```
x, y, z = [1, 2, 3]
first, *rest = [1, 2, 3, 4]       # rest = [2, 3, 4]
first, *mid, last = [1, 2, 3, 4]
a, b = b, a                       # swap
a[i], a[j] = a[j], a[i]           # swap elements
```

Comprehensions

```
[x*2 for x in a]                      # transform
[x for x in a if x > 0]               # filter
```

```
[x if x > 0 else 0 for x in a]         # if/else goes BEFORE for
[(x, y) for x in a for y in b]         # nested loops
[v for row in grid for v in row]       # flatten 2D
```

Sorting

```
a.sort(key=lambda p: p[1])             # by 2nd element
a.sort(key=lambda p: (p[0], -p[1]))    # multi-key, mixed direction
sorted(words, key=str.lower)           # case-insensitive
sorted(range(n), key=lambda i: a[i])   # argsort: indices ordered by value
# sort is stable; mixed types (int and str) raise TypeError
```

Built-ins on lists

```
len(a); sum(a); min(a); max(a)
any(a); all(a)                     # any truthy / all truthy
enumerate(a, 1)                    # (index, item), index starting at 1
zip(a, b)                          # pairs, stops at the shortest
zip(*matrix)                       # transpose
max(a, key=len)                    # item with the longest length
a.index(min(a))                    # position of the smallest
```

List recipes

```
list(dict.fromkeys(a))                          # dedupe, keeps order
sorted(set(a))                                  # dedupe, sorted
[a[i:i+n] for i in range(0, len(a), n)]         # chunks of n
[x for x in a if x != target]                   # remove ALL of a value
max(range(len(a)), key=a.__getitem__)           # index of the max
```

3. Dicts, sets, tuples

```
# DICT
d = {}
d[k] = v
d.get(k, 0)                       # default if missing
d.setdefault(k, []).append(x)     # create-if-missing then use
d.pop(k, None)                    # remove without KeyError
for k, v in d.items(): ...
d.keys(); d.values()
sorted(d.items(), key=lambda kv: kv[1])   # sort by value
k in d                            # O(1)
{k: v for k, v in pairs}          # comprehension

# SET (unordered, unique, O(1) membership)
s = set(a)
s.add(x); s.discard(x)            # discard never raises; remove does
s | t   # union          s & t   # intersection
s - t   # difference     s ^ t   # symmetric difference
s <= t  # subset
frozenset(a)                      # hashable set, usable as a dict key

# TUPLE (immutable, hashable)
p = (x, y)
seen = {(r, c)}                   # tuples work as keys; lists don't
```

- Sets and dicts can't hold lists. Convert to tuples first.
- Dicts keep insertion order (3.7+).
- Iteration order of a set is arbitrary; sort if the output depends on it.

4. math

```
import math
```

```
# CONSTANTS
math.pi; math.e; math.tau; math.inf; math.nan

# ROUNDING
math.floor(x); math.ceil(x); math.trunc(x)    # all return int
round(x)                          # built-in, banker's rounding: round(2.5) = 2
round(x, 2)

# POWERS / ROOTS
math.sqrt(x)                      # float; ValueError if x < 0
math.isqrt(n)                     # EXACT integer floor sqrt
math.cbrt(x)                      # 3.11+
math.exp(x)

# LOGS
math.log(x); math.log2(x); math.log10(x)
math.log(x, base)                 # can be off by float error; prefer log2/log10

# TRIG (radians!)
math.sin; math.cos; math.tan; math.asin; math.acos; math.atan
math.atan2(y, x)                  # angle of (x, y), all quadrants
math.radians(deg); math.degrees(rad)

# DISTANCE
math.hypot(3, 4)                  # 5.0
math.dist(p, q)                   # Euclidean distance between two points

# INTEGER / COMBINATORICS
math.factorial(n)
math.comb(n, k)                   # n choose k
math.perm(n, k)                   # ordered arrangements
math.gcd(a, b, ...)               # multi-arg 3.9+
math.lcm(a, b, ...)               # 3.9+
math.prod(it)                     # product of items

# SUMS / COMPARISON
math.fsum(it)                     # precise float sum
math.isclose(a, b)                # float compare; add abs_tol=1e-9 when comparing to
0
math.isnan(x); math.isinf(x); math.isfinite(x)

# MODULO
-7 % 3          # 2   (Python % follows the divisor)
math.fmod(-7, 3)  # -1.0 (sign follows the dividend, like C)
-7 // 2         # -4  (floor division)
```

- Never compare floats with `==`. Use `isclose`.
- `math.sqrt` on large ints loses precision. Use `isqrt`.
- `int(-2.7)` is `-2` (truncates); `-7 // 2` is `-4` (floors).
- `math.log(1000, 10)` gives `2.9999999999999996`. Use `log10`.
- Python ints are arbitrary precision, so no overflow, but huge numbers get slow.

5. collections

```
from collections import deque, Counter, defaultdict

# deque: O(1) at both ends. Use for BFS queues, never list.pop(0)
q = deque([start])
q.append(x); q.appendleft(x)
q.popleft(); q.pop()
deque(maxlen=k)                  # sliding window, old items fall off
q.rotate(1)                      # rotate right by 1

# Counter: frequency map
c = Counter("banana")            # {'a': 3, 'n': 2, 'b': 1}
c.most_common(2)                 # [('a', 3), ('n', 2)]
c["z"]                           # 0, missing keys don't raise
c1 + c2; c1 - c2                 # add / subtract counts

# defaultdict: auto-creates missing keys
g = defaultdict(list)            # adjacency list
g[u].append(v)
cnt = defaultdict(int)           # cnt[x] += 1 with no key check
```

6. heapq (min-heap)

```
import heapq
h = []
heapq.heappush(h, (dist, node))  # tuples compare left to right
d, u = heapq.heappop(h)          # smallest item
heapq.heapify(lst)               # list -> heap in O(n)
heapq.nlargest(3, lst)           # top k (nsmallest too)
heapq.heappush(h, -x)            # max-heap trick: negate, and negate back on pop
h[0]                             # peek at the smallest
```

- Ties compare the next tuple element. If those can be non-comparable (e.g. objects), add a counter as a tiebreaker: `(priority, count, item)`.
- There is no decrease-key. Push a new entry and skip stale ones on pop.

7. bisect (binary search on a sorted list)

```
import bisect
a = [1, 3, 3, 5, 8]
bisect.bisect_left(a, 3)         # 1, first index with a[i] >= 3
bisect.bisect_right(a, 3)        # 3, first index with a[i] > 3
bisect.insort(a, 4)              # insert, keeps sorted (O(n) insert)

# count of x:          bisect_right(a, x) - bisect_left(a, x)
# items < x:           bisect_left(a, x)
# items <= x:          bisect_right(a, x)
```

```
# largest item <= x:   a[bisect_right(a, x) - 1]   (check index >= 0)
# smallest item >= x:  a[bisect_left(a, x)]        (check index < len)
# key= parameter is 3.10+
```

The list must already be sorted, or results are garbage.

8. itertools

```
from itertools import (permutations, combinations, product, accumulate,
                       combinations_with_replacement, groupby, chain,
                       islice, count, pairwise)

permutations(a)                 # all orderings (n!)
permutations(a, 2)              # ordered pairs
combinations(a, 2)              # unordered pairs, no repeats
combinations_with_replacement(a, 2)
product(a, b)                   # cartesian product
product("01", repeat=3)         # all 3-bit strings
accumulate(a)                   # prefix sums
accumulate(a, max)              # running max (any binary function)
groupby(sorted_a)               # runs of equal items (sort first!)
chain(a, b); chain.from_iterable(grid)
islice(it, k)                   # first k items
pairwise(a)                     # (a0,a1), (a1,a2), ... (3.10+)
```

Results are lazy iterators. Wrap in `list()` if you need to reuse them.

9. functools

```
from functools import lru_cache, cache, reduce

@cache                          # memoize (3.9+), unbounded
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

@lru_cache(maxsize=None)        # same, works on older Python
def f(i, j): ...                # arguments must be hashable (no lists)

fib.cache_clear()               # reset between test cases
reduce(math.gcd, a)             # gcd of a whole list
```

10. Built-ins and bit tricks

```
divmod(a, b)                    # (a // b, a % b)
pow(a, b, mod)                  # fast modular exponentiation
pow(a, -1, mod)                 # modular inverse (3.8+, must be coprime)
abs(x); min(a, b, c); max(a, key=...)
sorted(a, key=..., reverse=True)
int(s, 2); bin(x)[2:]; hex(x); oct(x)

# BITS
x & 1                           # odd check
x >> k & 1                      # k-th bit
x | (1 << k)                    # set bit k
x & ~(1 << k)                   # clear bit k
x ^ (1 << k)                    # toggle bit k
```

```
x & (x - 1)                     # clear lowest set bit
x & -x                          # isolate lowest set bit
x.bit_count()                   # popcount (3.10+); else bin(x).count("1")
x.bit_length()                  # bits needed
(x & (x - 1)) == 0              # power of two (x > 0)

# SUBSET ENUMERATION
for mask in range(1 << n):
    for i in range(n):
        if mask >> i & 1: ...
```

Other imports

```
from fractions import Fraction   # exact rationals
from decimal import Decimal, getcontext   # exact decimals; getcontext().prec = 50
import random                    # random.shuffle(a) before sorting dodges anti-sort
hacks
import string                    # string.ascii_lowercase, string.digits
from operator import itemgetter  # a.sort(key=itemgetter(1)), faster than lambda
from array import array          # compact typed array
import time                      # time.perf_counter() for local timing
```